

18-646 Project Midterm Report

Student: Siddarth Kunisetty, Kushaan Misra, Adrian Muñoz

1 Computation Problem and Algorithm

Our proposed project is the design and GPU-accelerated implementation of a real-time Canny edge detection pipeline in images, which is part of a fundamental process in computer vision. Image edge detection is essential for tasks from object recognition to autonomous driving, and identifies boundaries after calculating image intensity gradients.

The Canny Edge Detection algorithm consists of multiple stages with each presenting unique GPU optimization challenges:

1. Gaussian Blur: Uses a 2D separable convolution with a configurable kernel to denoise the image. This stage is heavily parallelizable, and would be best exploited by shared memory tiling and memory-coalesced access.
2. Gradient Computation: Applies Sobel operators in horizontal and vertical directions to get the derivative of the pixels in both directions. This is necessary to determine the magnitude and orientation of the gradient for every pixel.
3. Non-maximum Suppression: Thins the detected edges by suppressing non-maximal pixel values along the gradient direction, which requires inter-pixel processing and management.
4. Double Thresholding and Hysteresis: Classifies pixels as strong, weak, or non-edges with the use of two thresholds. It then links edges based on connectivity. This linking behavior introduces data-dependent control flow that requires careful handling of warp divergence.

There will be many design decisions that we will need to make, such as how to parallelize the work in an efficient manner, thread block sizes, how to properly use shared memory, pipelining, etc.

2 Performance Baseline

To determine our performance baseline, we use the C++ implementation of Canny Edge Detection in OpenCV, which runs through the `Canny` function that the library provides. This function is run sequentially and is meant to run on a CPU. Due to the difficulty in getting OpenCV on the ece machines and the sheer size of our data set, we ran this code locally, on an ARM M2 processor. Although we do expect there to be some difference between the performance measured by the ARM M2 processor and the Intel x86 processors on the ece machines, we believe this is an adequate baseline performance to compare against, as our final project will be executing on a completely different architecture, i.e. a GPU.

To come up with a benchmark implementation, we load 200 small and 200 large images using C++, and run the Canny Edge Detection pipeline on these images. The small images have a resolution of 480 by 320 pixels, while the large images have one dimension that is 2040 pixels, while the other is an average of 1400 pixels. The small images were obtained through the Berkeley Segmentation Data Set and Benchmarks 500 (BSDS500 dataset). We use 200 of these 500 images as our benchmark for small images. For large images, we used 200 images from the Kaggle Div2K High Resolution Image dataset from Kaggle.

Before we provide a plot of the execution times of all of the images in our sampled datasets, we first provide average execution times for the edge detection pipeline on the small and large images. For small images, the average execution time is **612.3 us**, and for large images, the average execution time is **8.81 ms**.

We provide a scatter plot of execution times for all 200 of the small and large images on the ARM M2 architecture below.

Execution Time of Edge Detection Pipeline on Images

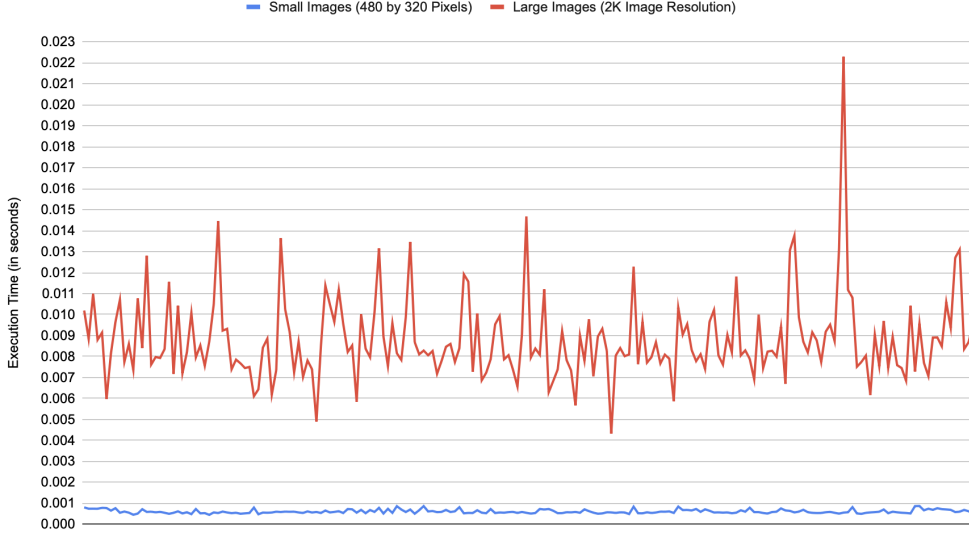


Figure 1: Scatter plot of execution times of edge detection pipeline on small and large images.

There are a few properties of this graph that are worth noting. First, the variance in the execution time of the edge detection pipeline on the large images is far higher than the variance seen in the execution time on the smaller images. This can be attributed to the variation in the sizes of the images that we see in the Div2K dataset, which is not uniform in the same way as the BSDS500 dataset.

Furthermore, it can also be seen that the average execution time seems to vary linearly with the number of pixels, which is to be expected. If we take the ratio of average execution times between the large and small images, we obtain $\frac{8.81}{.612} = 14.4$, which is very similar to the ratio in the absolute number of pixels present in the images.

3 Parallelization Strategy

3.1 Fused Kernel

We have split the algorithm into 4 steps. One observation is that the first 3 steps (blur, gradient, NMS) can be calculated independently in a subsection of the image (tile). For a given pixel at (i, j) , you need the tile of $[i - 2 : i + 2, j - 2 : j + 2]$ to calculate the Gaussian Blur. For the Sobel operator, you need the blurred pixels of $[i - 1 : i + 1, j - 1 : j + 1]$, which is a receptive field of $[i - 3 : i + 3, j - 3 : j + 3]$, and NMS requires no extra receptive field.

This means that if we split an image into tiles, each of these tiles can calculate the first 3 steps independently, and reuse the values of the previous step, without requiring inter-tile synchronization or communication. This allows us to fuse these kernels, which will remove 2 kernel launch overheads. Since we can store results in shared memory, we also remove 4 intermediate round trips to global memory, if we had launched 3 separate kernels.

One caveat is that there is some redundant computation done on tile edges. Given tile size T_x, T_y , the extra computation is $O(\frac{6*(T_x+T_y)}{T_x T_y})$. With large enough tile-size, this overhead becomes negligible when compared to kernel launch overhead and gmem latency. Memory will be stored in row-major order, in both global and shared memory.

3.2 Kernel Design

Each thread block will correspond to a tile ($T \times T$). The first step will be to load the $(T+6, T+6)$ “supertile” into shared memory. We will use a 32×32 threadblock size, to maximize the 1024 thread limit. These threads can stride across the supertile, potentially loading multiple values into shmem. They will sync before doing the blur step, guaranteeing all the values are in shmem.

For the blur, each thread will correspond to the pixels at $(tid, tid + 1024, \dots)$. All 32 threads in a warp will correspond to consecutive output pixels, and since they will step through the same “offsets” in the 5×5 Gaussian filter, they will always operate on consecutive shmem addresses causing no bank conflicts. They will write their

results to a secondary buffer in shmem for outputs. Since, at any given time, you only need one buffer for previous stage outputs and one buffer for the outputs, we can simplify the shared memory requirement. We follow this pattern for the gradient and NMS steps. We will also do the double thresholding section of hysteresis here, since it is “free” once we have gradient information. After this, we write the results back to gmem to persist them after the kernel ends.

Parallelizing this way allows us to make use of coalesced memory (since we have continuous pixel ranges), and shared memory allows us to avoid expensive trips to global memory. The Tesla T4 has 64KB shared memory per SM, and at our largest shared memory point, we use 1 fp16 output buffer (blur) and 2 fp16 output buffers (gradient values), so we can calculate an optimal tile size of 99x99 to maximize shmem utilization. We solve for the maximum T s.t. $2 * (T + 6)^2 + 2 * 2 * (T + 2)^2 < 64K$. Simply put, the size of our buffers should not exceed 64KB of shared memory. Our occupancy is limited by resident-threads/SM (1024 on a T4), so we can maximize shmem usage without hurting occupancy.

3.3 Idle Core Usage

The final hysteresis step does not exhibit parallelism as well as the previous steps. At this point, we have many “masked out” pixels, so naively parallelizing over all the pixels is wasteful. Additionally, the operation of “propagating” strong pixels over their neighbors is essentially a graph traversal. Graph traversals have a few properties which make them not ideal on GPUs, namely non-coalesced/varied memory accesses. Additionally, since these “weak chains” are non-uniform in length, you can have issues with thread divergence.

Our system has 32 CPUs, many of which will be idle doing just data movement. We also identify that graph traversal is better suited for CPUs, especially given the sparse nature of the graph (relative to the NxN image size), and that we have to transfer data back to CPU after finishing the algorithm. By doing the memcopy before hysteresis, we can use our idle cores to perform the graph traversal. We believe that due to the number of idle cores, we can get better utilization of both our hardware stacks. We will use streams to ensure that we are doing parallel image processing and keeping the GPU fed. One image will map to 1 stream, which will launch the appropriate number of thread blocks for the image size. On the CPU-side, we will have a work queue of images from which the cores can draw post-NMS images from.

It is possible that this intuition is incorrect (i.e., we are overestimating the slowdown from GPU bottlenecks or the CPU speed). We will profile this step to see if it is the bottleneck in our end-to-end latencies, and if so, will use a GPU-based implementation.

3.4 Design Summary

Our design is a mainly memory-bound problem, we have lots of data movement for relatively little computation. On the CPU side, we should utilize our cores as much as possible and optimize for things they specialize in (e.g. graph traversal). On the CUDA side, we should optimize for shmem usage. Additionally, we want our GPU to be fed, so we use streams to support input-level concurrency and give us enough thread blocks on small image inputs (i.e. Berkeley dataset).

4 Work Split

Adrian: Image frontend, data preprocessing (i.e. grayscale), cuda stream management, kernel launches (async etc...) -33%

Kush: Fused kernel implementation -33%

Sid: Image “backend”, CPU hysteresis, work queue -33%

Note: We interpreted the “kernel analysis” as general analysis of the parts of our project that are being parallelized, and not the literal CUDA kernel, hence how we divided the above work. We also plan to help each other in case one section finishes earlier, so we anticipate equal work contribution.